

画像解析・対話ツール

1、はじめに

本ツールは、GeminiとGroqのAPIを活用し、ユーザーがアップロードした画像に対して OCRを行い、その結果についてLLMへ 質問することができるWebアプリです。

チャット形式のため、画像の説明だけでなく気になる点を深掘りして対話が可能です。また、OCR済みの画像とテキストデータはローカルデータベースに格納され、以降の回答における知識として提供されます。

開発動機

私は学校や自己学習を通じて、OCRとチャットボットの アプリをそれぞれ制作した経験があります。今後のキャリアアップを見据え、単一機能のアプリ 制作に留まらず、複数の技術を統合することでさらなるスキルアップに繋がると考えました。

そこで、OCRの抽出結果をチャットボットの知識データとして活用する画像解析・対話ツールを開発することになりました。併せて、未経験のサービスも積極的に取り入れることで、扱える技術の幅を広げることができると考えました。

使用する言語・ライブラリ

- ・Python
- ・HTML+css
- ・JavaScript
- ・flask
- ・dotenv
- ・google-generativeai
- ・chromadb
- ・uuid

2、画面・使い方

アプリの画面はこちらになります。機能ごとに分けて説明します。



・画像アップロードエリア(左上)

ユーザーは画像を「画像をアップロード」エリアをクリックまたは画像ファイルをドラッグすることで、画像ファイルをアップロードできます。accept="image/*"を指定しているため、画像以外のファイルが選択されるのを防いでいます。画像がアップロードされるとJavaScriptの関数が呼び出され、自動的にOCRが実行されます。

・OCR結果表示エリア(左下)

OCRの結果が返されると、プログラムはその文字列を「解析が完了すると、ここにテキストが表示されます」エリアのtextContentに代入して表示します。クォータ不足などの理由で解析できなかった場合は、「Geminiに接続できませんでした」と表示されます。

・チャットボットエリア(右)

画像がアップロードされOCRの結果が表示されると、画面右側のチャットボットエリアにて、その内容についてGroqを用いた質問や対話が可能になります。なお、画像のアップロードがない場合や、入力が空の状態での送信には反応しない仕様となっています。

・データリセットボタン(右上)

クリックすることで、アップロードしている画像・チャット記録はリセットされます。

3、システムの流れ

システムの流れをパートごとに説明します。

1、画像アップロード

まず、ユーザーが画像をアップロードしなければ以降の機能は使えないようになっています。画像がアップロードされるとJavaScript関数が呼ばれて、アップロードエリアにその画像を表示し、アップロード成功をユーザーに伝えます。その後、別の関数でFormDataオブジェクトを生成して画像を保存し、fetch関数でPython側の関数を呼び出し、画像をプロンプトとともにGemini APIに渡してOCRを実行します。

2、OCR結果表示

OCRの結果が返ってきたら、その内容を結果表示エリアのtextContentに代入して表示します。改行も含めて表示するため、テキストが長い場合にはスクロールできるようにしています。また、結果のテキストはデータベースに保存され、以降の知識データとして使えます。

3、チャットボット

OCRが成功して表示されると、チャットエリアに「画像をアップロードしました。対話を始められます。」と表示され、Groqを通じたチャット形式で画像についての質問や対話ができるようになります。リセットやセッション終了まで会話履歴をDBに保存し、プロンプトと共に送信することで、パーソナライズされた対話を実現します。画像専用のため、無関係な内容には、「画像に関係ない質問は答えられません」と返答するようにしています。また、OCR結果エリアと同じく、チャットが多くなるとスクロールできるようにしています。

4、データリセット

ボタンを押すと、JavaScript関数でページを初期に戻し、別の関数を呼び出してデータベースからチャット記録を全て消します。

実際に使ってみたらこんな感じです。



4、苦労点・工夫した点

作成時の苦労点・工夫点はこちらです。

- ・今回のプログラムではAPIを呼び出す機能が二つありますが、同じAPIを使い回すのではなく、OCRに優れたGeminiと、チャットに長けたGroqを組み合わせるハイブリッド方式にすることで、システムの精度を最大化しました。

- ・初めてデータベースを利用しましたが、セキュリティと利便性を考慮して、ローカルでも手軽に扱えるChromaDBにしました。将来の公開も見据え、どの環境でもフォルダを作成してデータを保存できるChromaDBのメリットを考慮したわけです。

- ・無料のAPIを使用しているため、トークン制限を超えないよう、プロンプトの長さを最小限にしました。特にGroqの呼び出しでは、知識データをすべて含めるのではなく、ChromaDBのquery関数でユーザーの質問に近いテキストデータや会話履歴を5件だけ抽出してプロンプトに入れることで、無駄な知識を渡すことを防いでいます。

- ・前のセクションで話した、ボタンを押すと会話履歴をリセットする理由としては、似た質問がデータベースに蓄積するのを防ぐためです。データが溜まると、query関数が過去の会話履歴ばかりを優先してしまい、知識を渡せなくなるのを避けています。また、ページをリフレッシュした際にも履歴を消去する関数が呼ばれるようにしています。

- ・今後の展望として、まずは有料のAPIを利用することで、トークンを気にせずアプリを使えるようにしたいです。また、ユーザー登録機能を実装し、ユーザーごとの特徴などをデータベースに保存することで、永続的なパーソナライズ会話を実現したいと考えています。